TRƯỜNG ĐẠI HỌC ……

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

MÔN: TRIỂN KHAI VÀ QUẢN TRỊ HỆ THỐNG

ĐỀ TÀI

Triển khai và quản trị hệ thống quản lý sinh viên

SINH VIÊN THỰC HIỆN

……

Lớp: ……..

Giảng viên hướng dẫn: ……..

……., tháng…… năm …….

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN DỰ ÁN VÀ KIẾN TRÚC HỆ THỐNG 3
1.1. Giới thiệu bài toán 3
1.2. Mục tiêu của dự án 3
1.3. Mô hình kiến trúc hệ thống 3
CHƯƠNG 2. TRIỂN KHAI HẠ TẦNG DỊCH VỤ VỚI DOCKER COMPOSE 5
2.1. Cấu hình Docker Compose 5
2.2. Cấu hình Nginx Reverse Proxy 5
2.3. Khởi tạo CSDL và Quản trị qua phpMyAdmin 5
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG GIÁM SÁT VÀ QUẢN LÝ LOG TẬP TRUNG 10
3.1. Giám sát tài nguyên hệ thống 10
3.2. Quản lý log tập trung 11
CHƯƠNG 4. QUẢN LÝ QUÁ TRÌNH PHÁT TRIỂN VỚI GIT VÀ GITHUB 13
4.1. Quy trình quản lý phiên bản 13
4.2. Đồng bộ mã nguồn lên GitHub 13
CHƯƠNG 5. KẾT QUẢ TRIỂN KHAI VÀ KẾT LUẬN 14
5.1. Kết quả đạt được 14
5.2. Kết luận và hướng phát triển 14

CHƯƠNG 1. TỔNG QUAN DỰ ÁN VÀ KIẾN TRÚC HỆ THỐNG

1.1. Giới thiệu bài toán

Trong bối cảnh phát triển phần mềm hiện đại, việc đóng gói và triển khai ứng dụng dạng microservices hoặc multi-container giúp tối ưu hóa khả năng đóng gói, độc lập môi trường và dễ dàng mở rộng. Bài toán đặt ra là xây dựng một **Hệ thống Quản lý Sinh viên** đầy đủ các chức năng CRUD (Quản lý Lớp học, Quản lý Sinh viên, Quản lý Điểm số).

Hệ thống đòi hỏi không chỉ vận hành ổn định mà còn phải đáp ứng các tiêu chuẩn khắt khe về an toàn bảo mật (SSL/TLS, cách ly mạng, phân quyền), cùng hệ thống giám sát tài nguyên (Monitoring) và thu thập nhật ký hoạt động (Centralized Logging) tập trung.

1.2. Mục tiêu của dự án

Triển khai ứng dụng Web Flask & Cơ sở dữ liệu MySQL: Đóng gói ứng dụng web quản lý sinh viên với giao diện Bootstrap hiện đại và CSDL MySQL, hỗ trợ quản trị qua phpMyAdmin.

Cấu hình Reverse Proxy & Bảo mật (Security Hardening): Đóng vai trò làm cổng giao tiếp duy nhất qua Nginx mã hóa HTTPS (SSL tự ký), áp dụng các Security Headers (CSP, X-Frame-Options, X-XSS-Protection) và thiết lập mô hình cách ly mạng (Network Isolation).

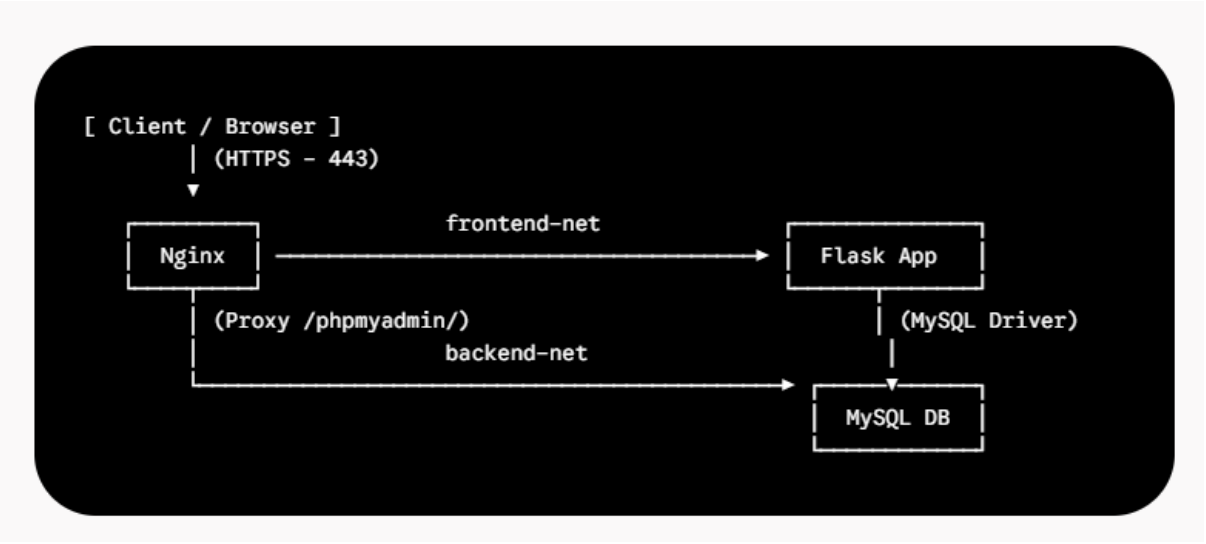
Xây dựng hệ thống Giám sát & Quản lý Log tập trung: Tích hợp Prometheus + cAdvisor để thu thập metrics hệ thống/container, Grafana để trực quan hóa biểu đồ, và Grafana Loki + Promtail để đẩy log tập trung.

Quản lý mã nguồn theo chuẩn Git/GitHub Workflow: Quản lý lịch sử phát triển dự án với các commit có ý nghĩa rõ ràng.

1.3. Mô hình kiến trúc hệ thống

Hệ thống được thiết kế theo kiến trúc Multi-container được điều phối bởi Docker Compose, phân chia thành 2 phân vùng mạng độc lập (Network Isolation):

- Phân vùng **frontend-net**: Dùng cho các dịch vụ tiếp nhận request công khai hoặc cần giao tiếp ngoài máy host:
  - Nginx: Proxy ngược tiếp nhận HTTPS (cổng 443) điều hướng đến Flask App và phpMyAdmin.
  - Prometheus & Grafana: Thu thập metrics và hiển thị dashboard.
  - Promtail & Loki: Thu thập và lưu trữ log
- Phân vùng **backend-net**: Phân vung nội bộ chứa tài nguyên nhạy cảm:
  - MySQL DB (Cổng 3306): Chỉ giao tiếp nội bộ Flask App và phpMyAdmin, không mở port trực tiếp ra máy host.



Hình 1.1. Sơ đồ kiến trúc tổng quan hệ thống Quản lý Sinh viên và luồng dữ liệu giữa các dịch vụ Docker

CHƯƠNG 2. TRIỂN KHAI HẠ TẦNG DỊCH VỤ VỚI DOCKER COMPOSE

2.1. Cấu hình Docker Compose

Toàn bộ hạ tầng được định nghĩa trong file docker-compose.yml, khai báo rõ các container, biến môi trường, ổ đĩa lưu trữ (volume) và hạ tầng mạng.

- Dịch vụ Core Web (app): Chạy ứng dụng Python Flask, áp dụng nguyên tắc Non-root Container trong Dockerfile nhằm hạn chế tối đa nguy cơ leo quyền nếu ứng dụng bị tổn thương
- Dịch vụ CSDL (db): Sử dụng mysql:8.0, dữ liệu được lưu bền vững qua volume db_data.
- Định nghĩa hai hạ tầng mạng trong file docker-compose.yml:

```yaml
networks:
  frontend-net:
    driver: bridge
  backend-net:
    driver: bridge
    internal: false
```

Hình 2.1. Định nghĩa hạ tầng mạng của hệ thống

```
dinhmanh@dinhmanh-VMware-Virtual-Platform:~/student-management-system$ docker compose ps
NAME                IMAGE                                COMMAND                  SERVICE      CREATED       STATUS                    PORTS
student_app         student-management-system-app        "python app.py"          app          6 hours ago   Up 2 hours                0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
student_cadvisor    gcr.io/cadvisor/cadvisor:latest      "/usr/bin/cadvisor -…"   cadvisor     6 hours ago   Up 2 hours (healthy)      8080/tcp
student_db          mysql:8.0                            "docker-entrypoint.s…"   db           6 hours ago   Up 2 hours                3306/tcp, 33060/tcp
student_grafana     grafana/grafana:latest               "/run.sh"                grafana      6 hours ago   Up 2 hours                0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
student_loki        grafana/loki:2.9.0                   "/usr/bin/loki -conf…"   loki         6 hours ago   Up 25 minutes             0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp
student_nginx       nginx:alpine                         "/docker-entrypoint.…"   nginx        6 hours ago   Up 2 hours                0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
student_phpmyadmin  phpmyadmin/phpmyadmin               "/docker-entrypoint.…"   phpmyadmin   6 hours ago   Up 2 hours                0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
student_prometheus  prom/prometheus:latest              "/bin/prometheus --c…"   prometheus   6 hours ago   Up 2 hours                0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
student_promtail    grafana/promtail:2.9.0               "/usr/bin/promtail -…"   promtail     6 hours ago   Up 25 minutes
```

Hình 2.2. Trạng thái khởi chạy thành công của tất cả các container dịch vụ

2.2. Cấu hình Nginx Reverse Proxy

Nginx được cấu hình để lắng nghe trên cổng 80 (tự động chuyển hướng HTTP → HTTPS) và cổng 443 với chứng chỉ SSL tự ký.

Bảo mật Security Headers trong nginx.conf:

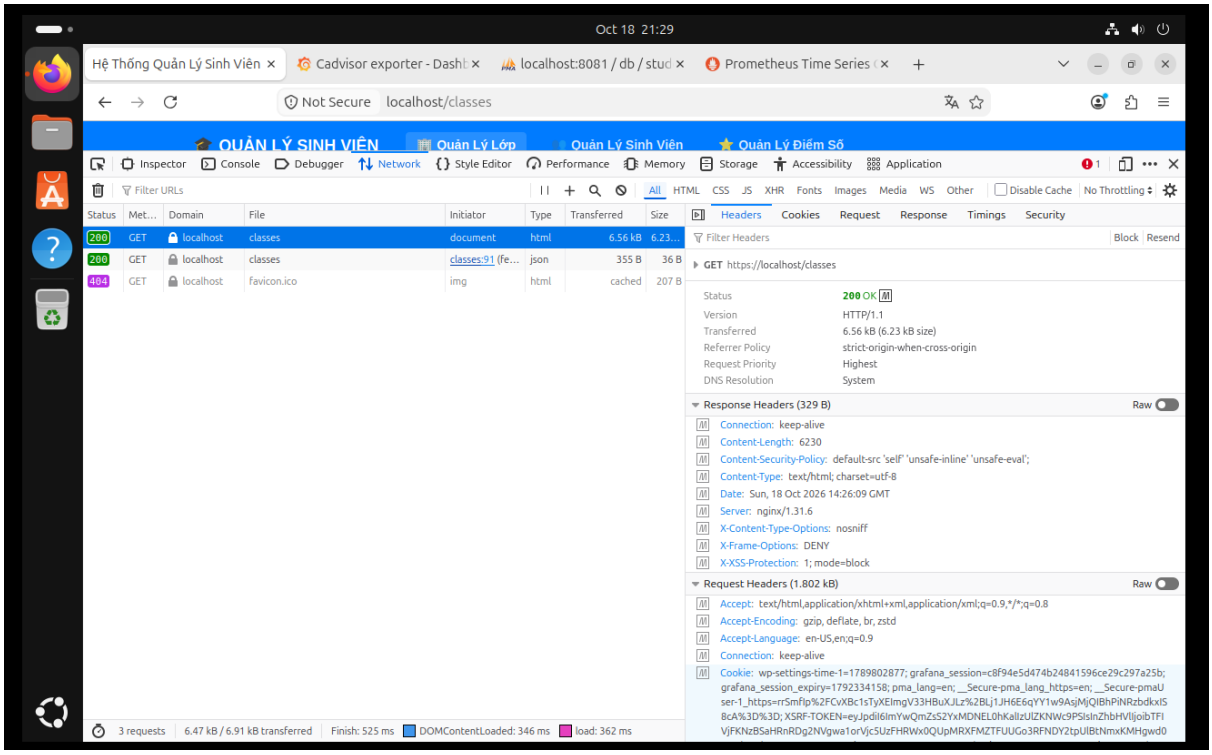
- X-Frame-Options "DENY": Chống tấn công Clickjacking
- X-Content-Type-Options "nosniff": Ngăn ngừa MIME-sniffing.
- Content-Security-Policy: Giới hạn các nguồn tài nguyên script/style được phép thực thi.

Định tuyến Proxy Pass: Chuyển tiếp truy cập trang chủ / sang http://flask_app:5000 và đường dẫn /phpmyadmin/ sang http://phpmyadmin:80

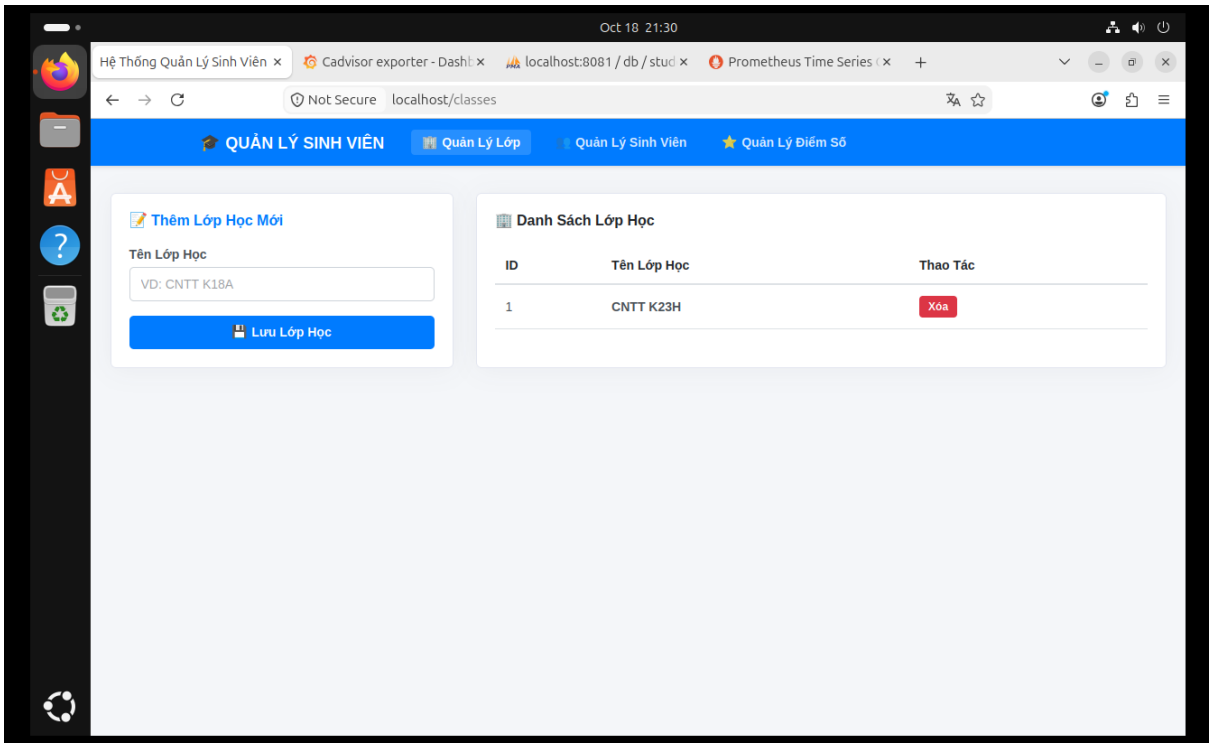
2.3. Khởi tạo CSDL và Quản trị qua phpMyAdmin

Cơ sở dữ liệu bao gồm 3 bảng có mối quan hệ ràng buộc khóa ngoại (Foreign Keys):

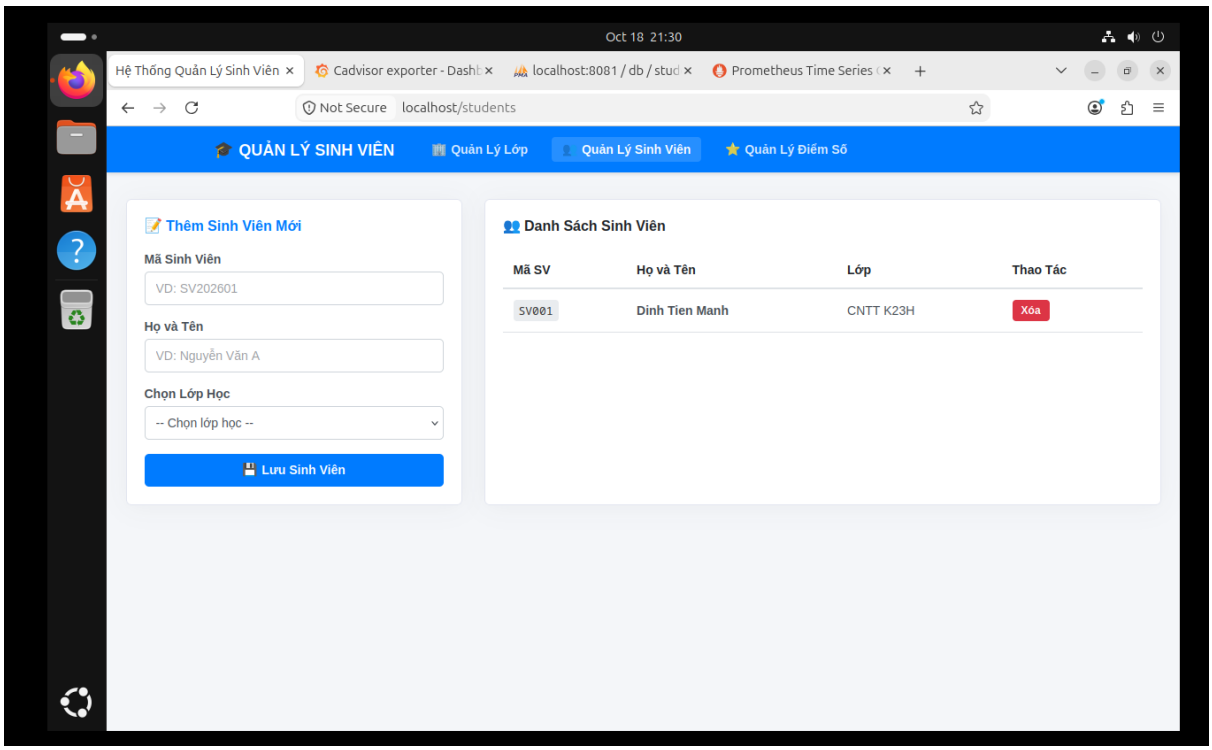
- classes (id, class_name)
- students (id, student_code, full_name, class_id)
- grades (id, student_id, subject_name, score)



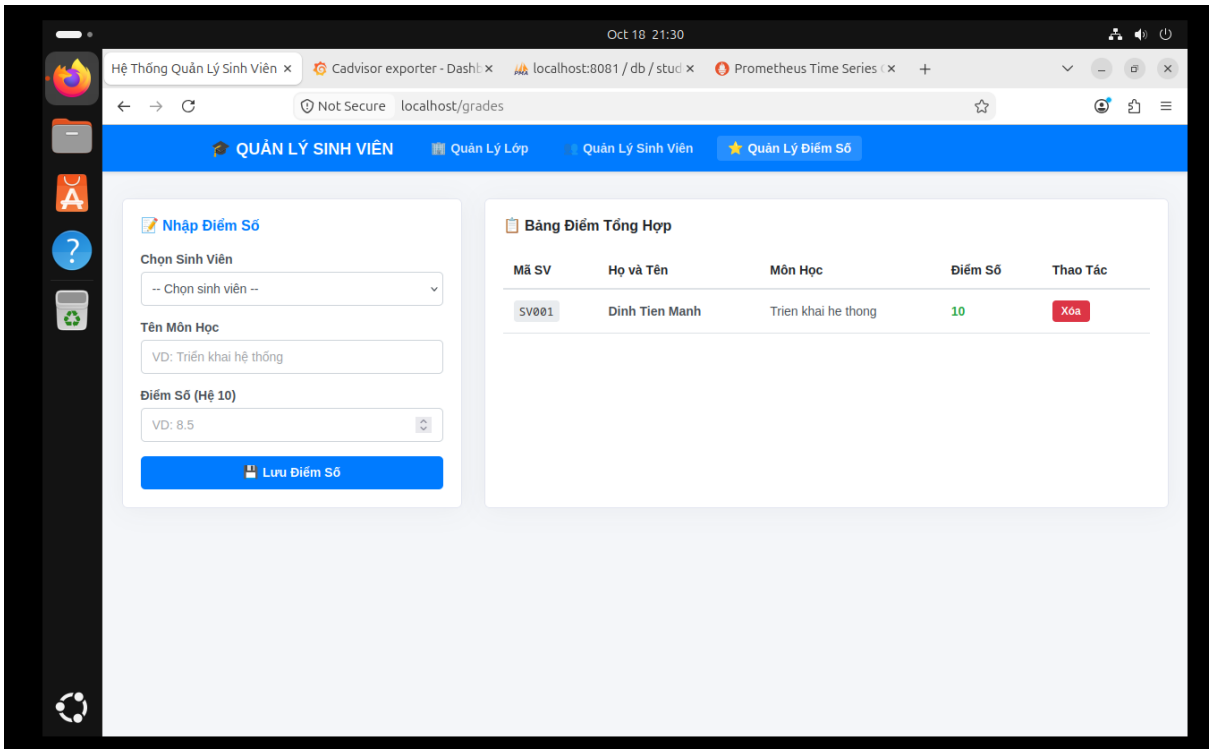
Hình 2.3. Kiểm tra các Response Headers bảo mật (Security Headers) do Nginx phản hồi trên Developer Tools (F12)



Hình 2.4. Giao diện màn hình Quản lý Lớp học và danh sách lớp học trên ứng dụng Flask



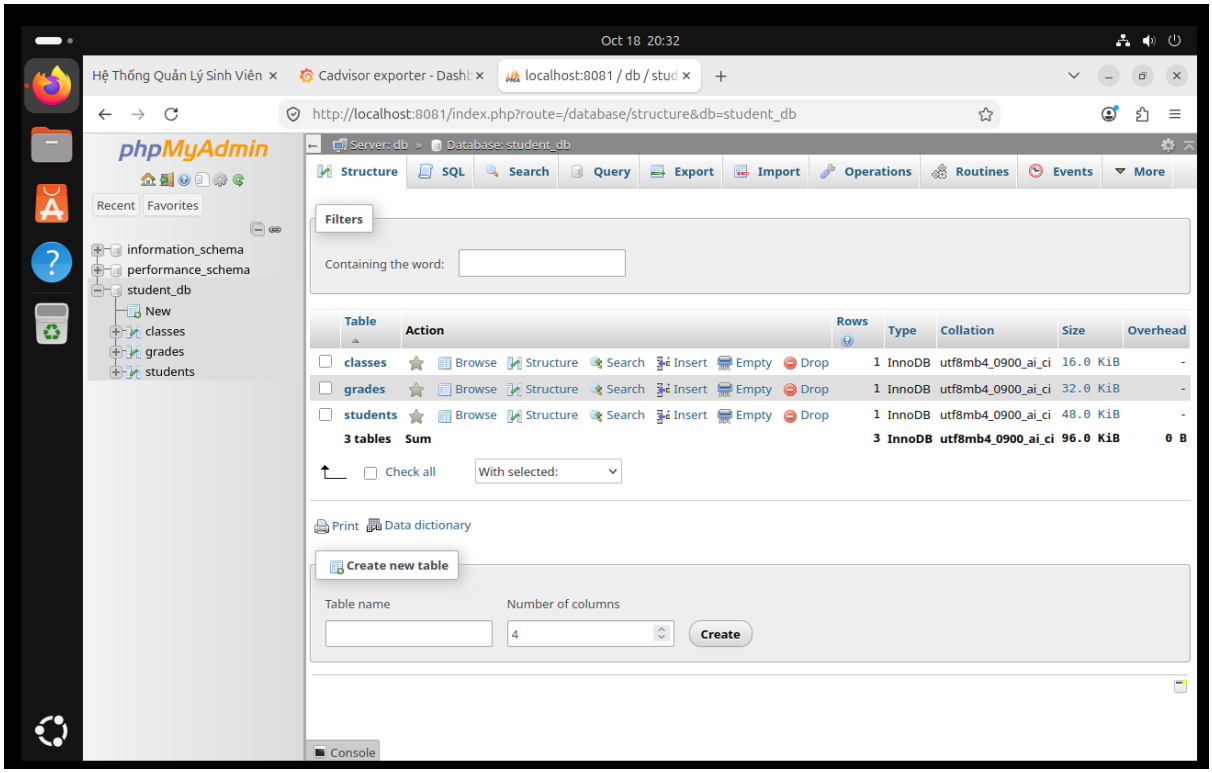
Hình 2.5. Giao diện màn hình Quản lý Sinh viên và form thêm sinh viên mới



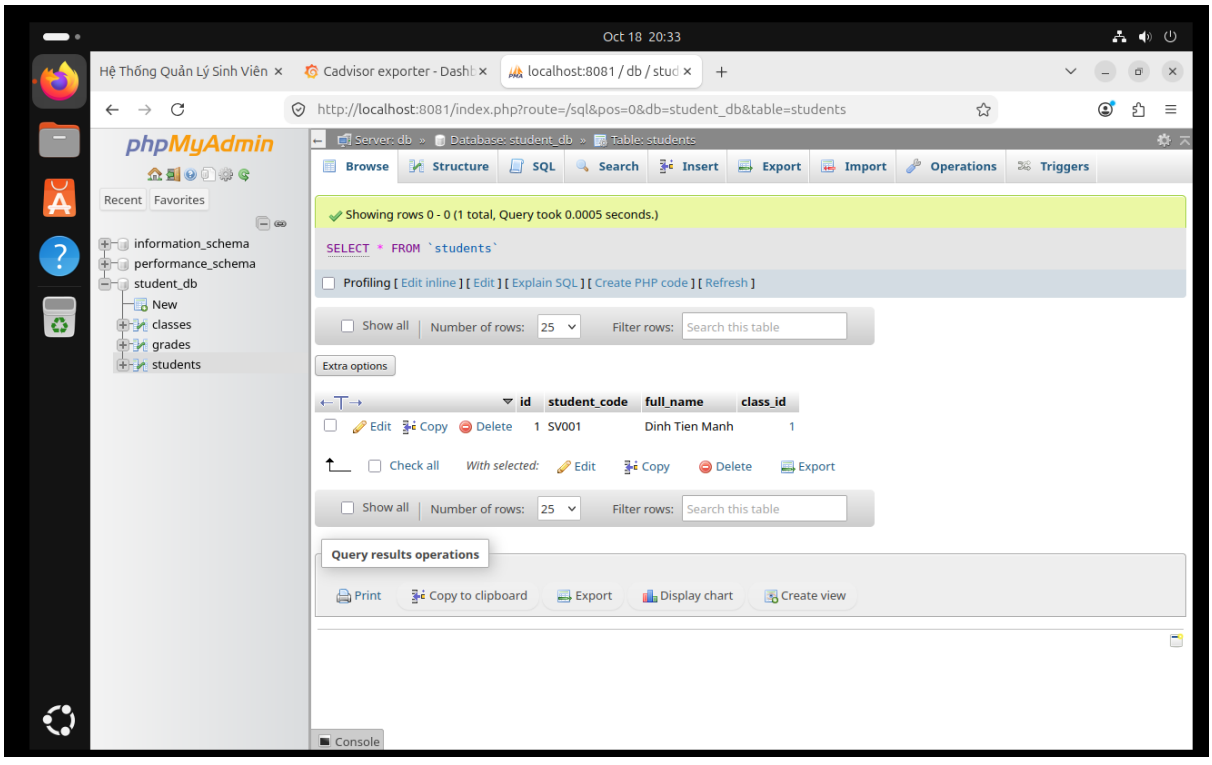
Hình 2.6. Giao diện màn hình Quản lý Điểm số và Bảng điểm tổng hợp

Ứng dụng tự động khởi tạo cấu trúc bảng khi khởi chạy thông qua hàm init_db(). Người dùng có thể trực quan hóa và thao tác trực tiếp với dữ liệu bằng phpMyAdmin

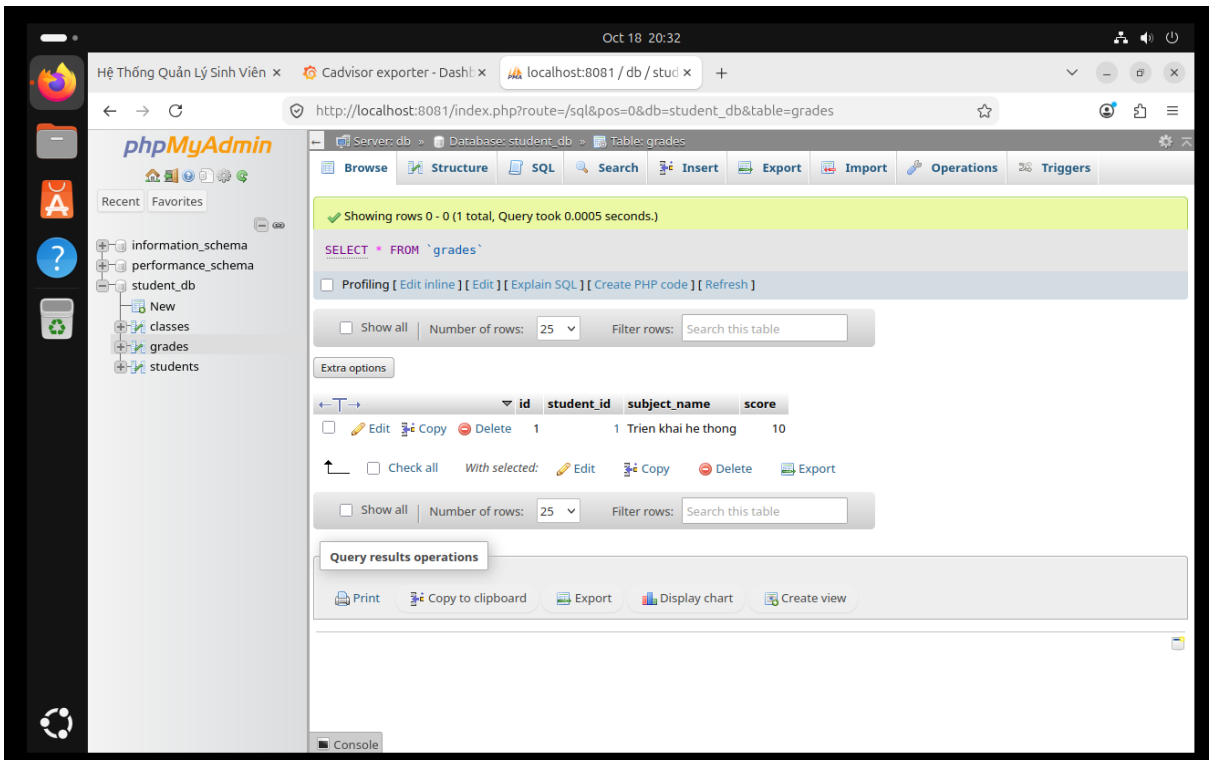
qua địa chỉ https://localhost/phpmyadmin/ hoặc http://localhost:8081



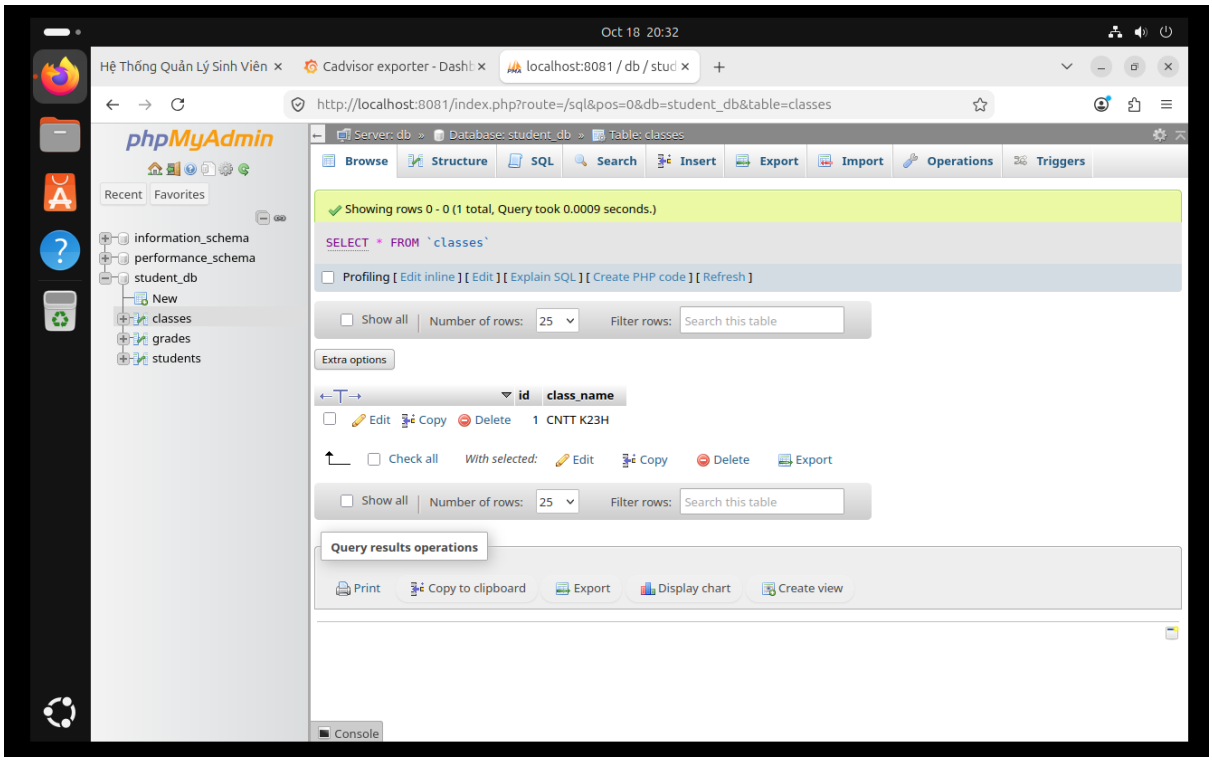
Hình 2.7. Giao diện quản trị cơ sở dữ liệu MySQL thông qua phpMyAdmin



Hình 2.8. Giao diện quản trị cơ sở dữ liệu MySQL thông qua phpMyAdmin



Hình 2.9. Giao diện quản trị cơ sở dữ liệu MySQL thông qua phpMyAdmin



Hình 2.10. Giao diện quản trị cơ sở dữ liệu MySQL thông qua phpMyAdmin

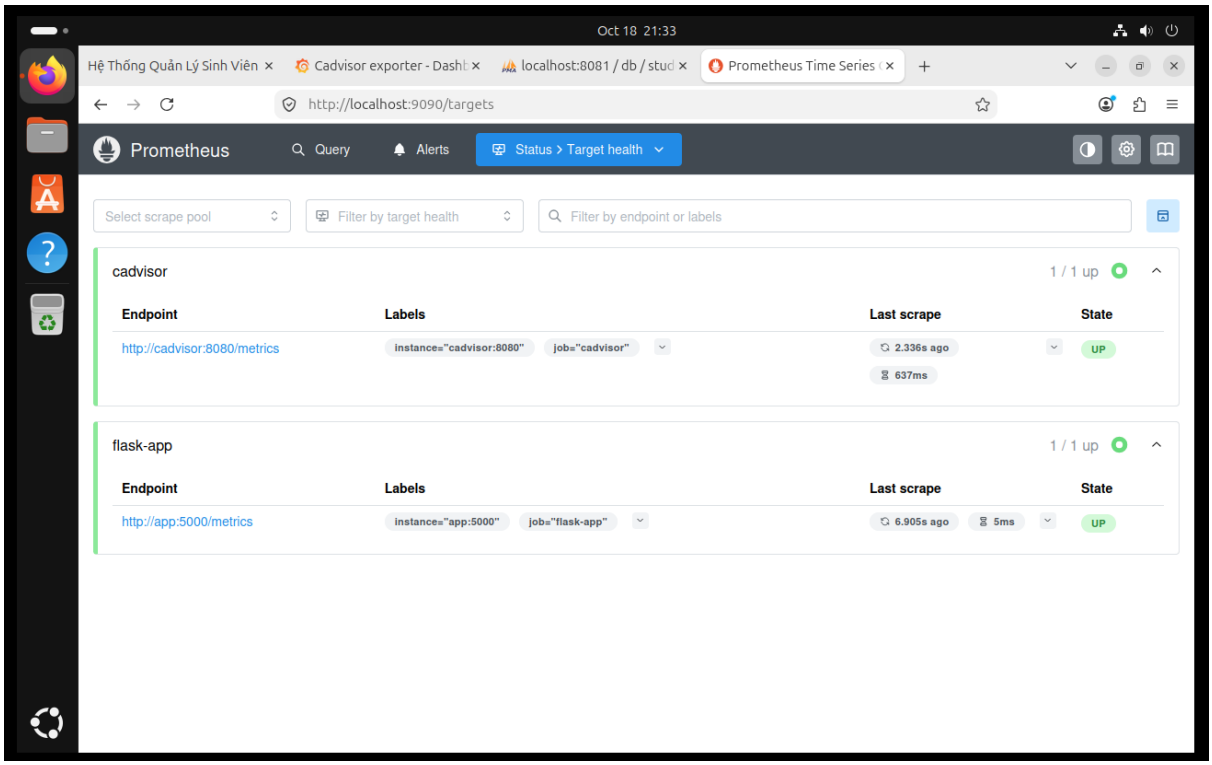
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG GIÁM SÁT VÀ QUẢN LÝ LOG TẬP TRUNG

3.1. Giám sát tài nguyên hệ thống

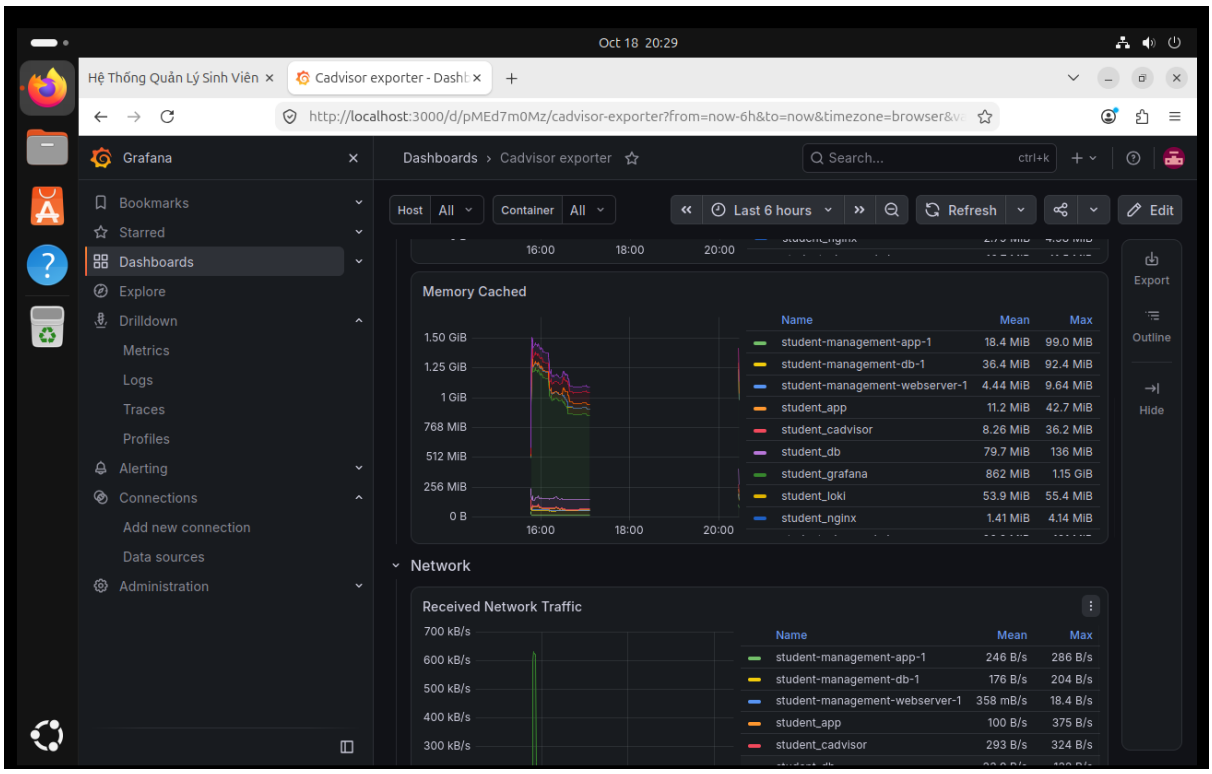
Sử dụng bộ đôi Prometheus và Grafana :

- **Prometheus:** Định kỳ 15 giây gửi HTTP GET request đến endpoint /metrics của Flask App và cAdvisor để thu thập thông số hoạt động.
- **cAdvisor:** Thu thập chỉ số tài nguyên phần cứng (CPU, RAM, Network I/O) của từng Docker Container đang chạy.
- **Grafana:** Khởi chạy tại port 3000, kết nối Data Source Prometheus để trực quan hóa biểu đồ tài nguyên theo thời gian thực.

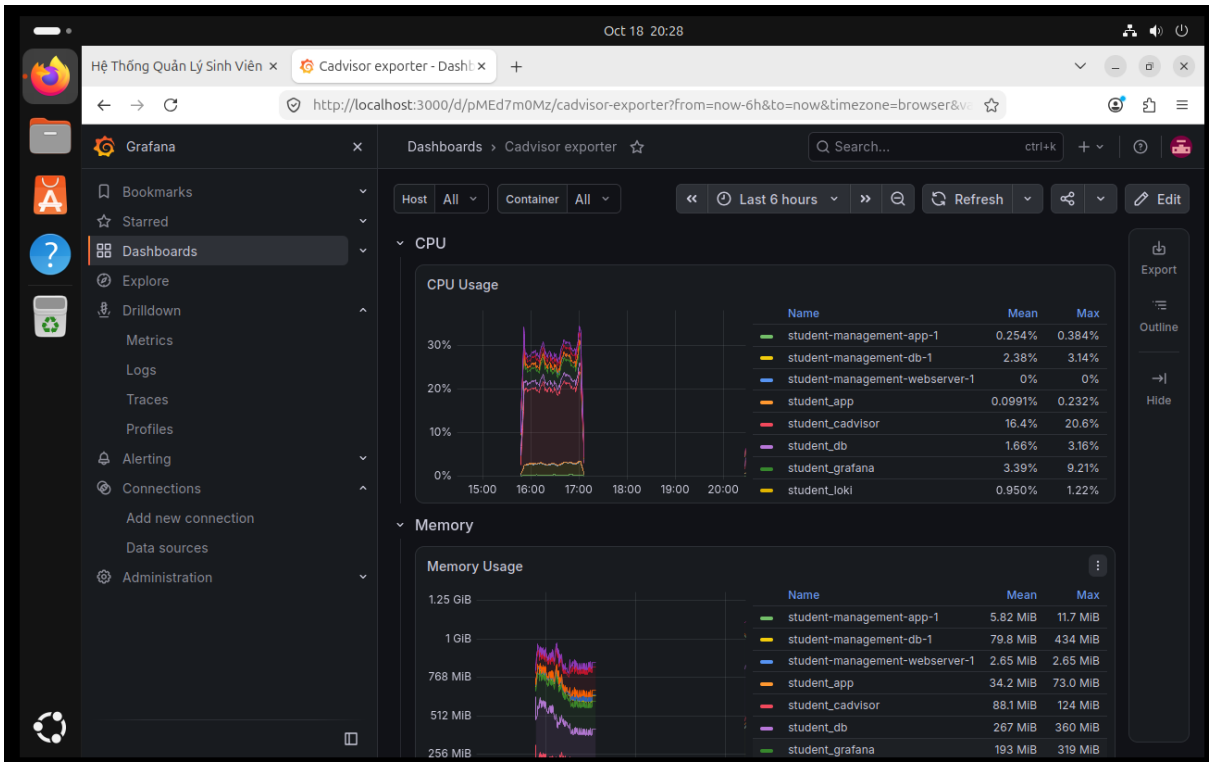
Trạng thái kiểm tra trên giao diện Prometheus tại http://localhost:9090/targets cho thấy các Target đều ở trạng thái **UP** (màu xanh), đảm bảo luồng thu thập dữ liệu diễn ra liên tục.



Hình 3.1. Trạng thái hoạt động (Health State) của các Target giám sát trên giao diện Prometheus



Hình 3.2. Dashboard trực quan hóa các chỉ số tài nguyên CPU, RAM và Network của các Container trên Grafana

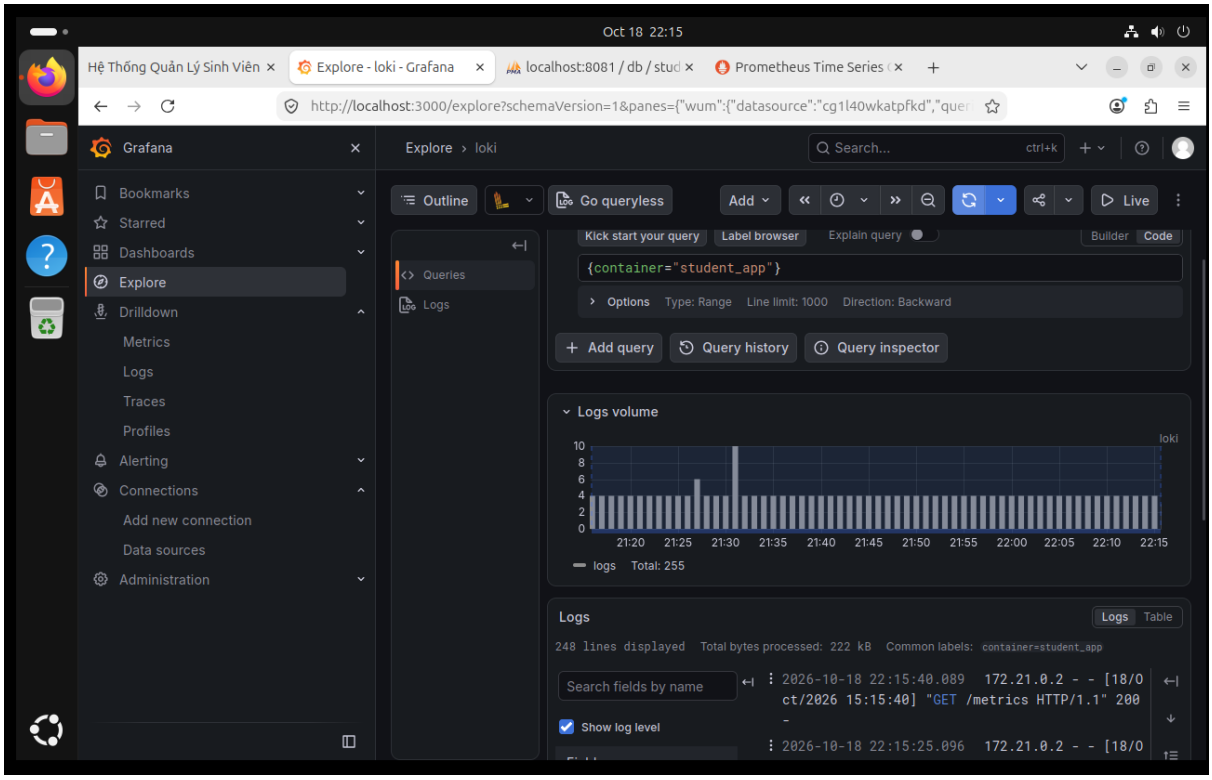


Hình 3.3. Dashboard trực quan hóa các chỉ số tài nguyên CPU, RAM và Network của các Container trên Grafana

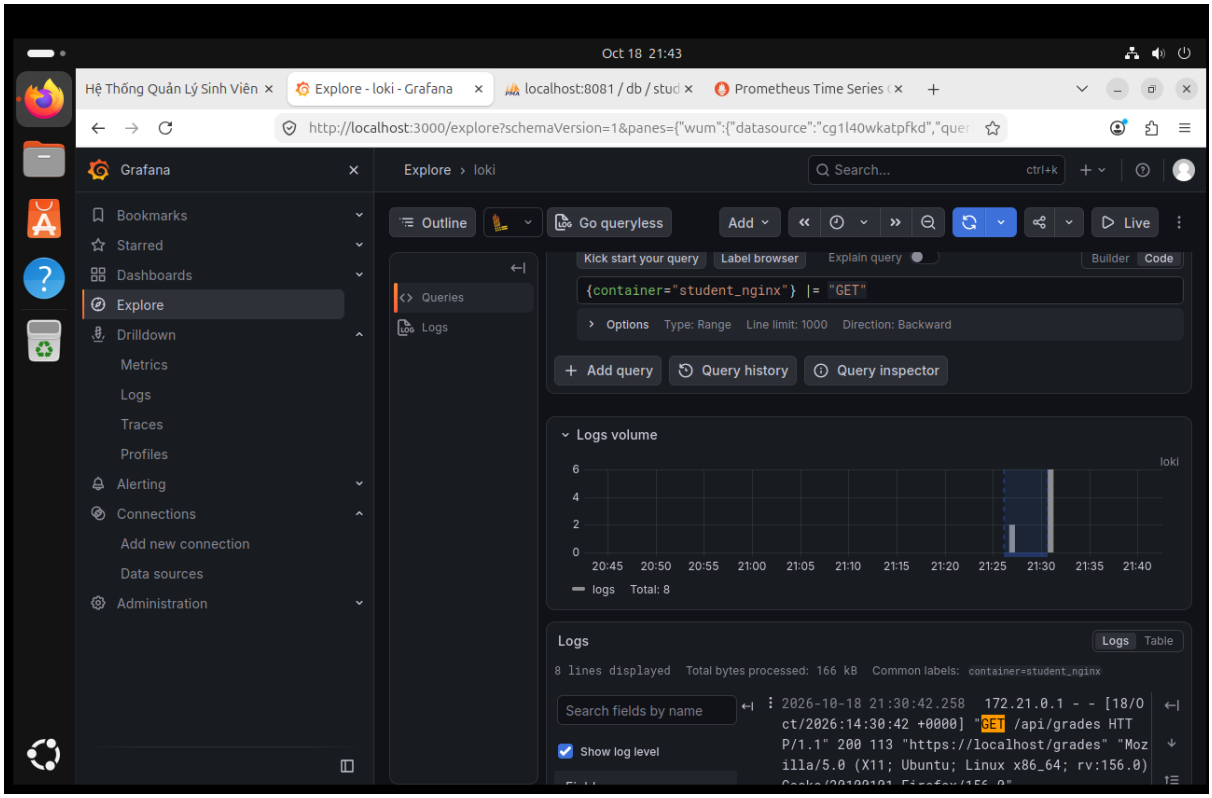
3.2. Quản lý log tập trung

Sử dụng bộ đôi **Grafana Loki** và **Promtail**:

- **Promtail:** Đóng vai trò là Agent mount trực tiếp ổ đĩa /var/lib/docker/containers trên máy host để đọc tất cả log xuất ra từ stdout/stderr của các container, sau đó gắn tag và đẩy (push) về Loki.
- **Loki:** Lưu trữ và đánh chỉ mục log theo label.
- Các truy vấn LogQL minh chứng trên Grafana Explore:



Hình 3.4. Kết quả truy vấn nhật ký ứng dụng Flask (App Logs) tập trung qua Grafana Loki



Hình 3.5. Kết quả truy vấn nhật ký hoạt động (Access Logs) của Nginx bằng ngôn ngữ LogQL trên Grafana Explore

CHƯƠNG 4. QUẢN LÝ QUÁ TRÌNH PHÁT TRIỂN VỚI GIT VÀ GITHUB

4.1. Quy trình quản lý phiên bản

Dự án áp dụng chặt chẽ quy trình Git Workflow, phân chia công việc thành các cột mốc có ý nghĩa rõ ràng thông qua lịch sử Commit:

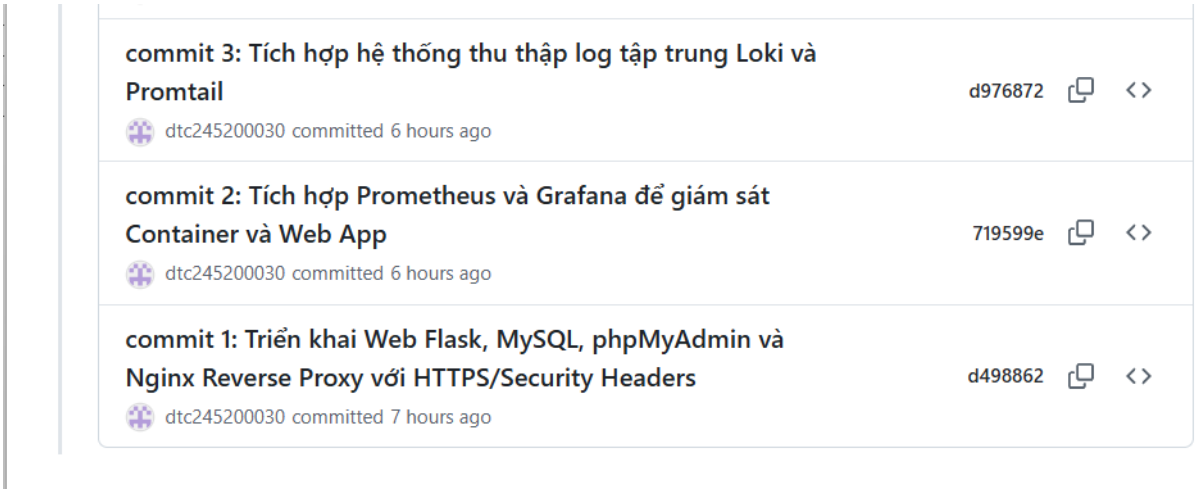
- **Commit 1:** Triển khai core ứng dụng Web Flask, MySQL DB, phpMyAdmin và cấu hình Nginx Reverse Proxy mã hóa SSL & Security Headers.
- **Commit 2:** Bổ sung hạ tầng giám sát Prometheus, cAdvisor và trực quan hóa Grafana.
- **Commit 3:** Tích hợp hệ thống thu thập log tập trung Loki + Promtail và hoàn thiện file README.md hướng dẫn vận hành.

4.2. Đồng bộ mã nguồn lên GitHub

Mã nguồn dự án được lưu trữ tại Repository Public trên GitHub.

File .gitignore được cấu hình đầy đủ để loại bỏ các tệp tin tạm, cache (__pycache__) và chứng chỉ SSL nhạy cảm.

File README.md trình bày chi tiết kiến trúc, danh sách công nghệ và hướng dẫn thao tác chạy hệ thống chỉ với một lệnh docker compose up -d.



Hình 4.1. Lịch sử Commit chi tiết (Commits History) thể hiện đủ 03 cột mốc phát triển dự án trên GitHub

CHƯƠNG 5. KẾT QUẢ TRIỂN KHAI VÀ KẾT LUẬN

5.1. Kết quả đạt được

Triển khai thành công ứng dụng Quản lý Sinh viên chuẩn 3 lớp (3-tier) hoạt động ổn định trên Docker Compose.

Giao diện người dùng Bootstrap đẹp mắt, hỗ trợ đầy đủ tính năng CRUD cho Lớp học, Sinh viên và Điểm số.

Đảm bảo an toàn bảo mật thông qua Nginx Reverse Proxy (SSL/TLS), Security Headers, Non-root Container và Network Isolation giữa Frontend/Backend.

Xây dựng hoàn chỉnh hệ thống giám sát Prometheus/Grafana và thu thập Log tập trung Loki/Promtail.

Quản lý mã nguồn minh bạch, tuân thủ đúng chuẩn 03 Commits trên GitHub

5.2. Kết luận và hướng phát triển

Kết luận: Dự án đã đáp ứng đầy đủ các yêu cầu kỹ thuật đề ra, minh chứng cho khả năng đóng gói, triển khai và quản trị hệ thống phần mềm chuyên nghiệp.

Hướng phát triển:

- Tích hợp quy trình CI/CD (GitHub Actions): Tự động hóa kiểm thử và deploy ứng dụng.
- Tích hợp cơ chế Phân quyền người dùng (RBAC): Xây dựng quản lý vai trò người dùng (Admin, Giảng viên, Sinh viên) và xác thực để kiểm soát truy cập các API CRUD
- Triển khai chứng chỉ SSL thực tế (Let's Encrypt): Thay thế cho SSL tự ký hiện tại để tăng độ bảo mật
- Mở rộng hạ tầng sang mô hình Docker Swarm hoặc Kubernetes: Tăng khả năng chịu tải và đảm bảo tính sẵn sàng cao (High Availability) cho hệ thống